

Bài tập trên lớp 27/04 – Môn Lập trình mạng

Họ và tên: Nguyễn Thu Trang

MSSV: 20235441

Bài 1: Chat server

Đề bài: Viết chương trình chat_server thực hiện các chức năng sau:

- Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp: **"client_id: client_name"**, trong đó client_name là tên của client, xâu ký tự viết liền.
- Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id "abc" gửi "xin chào" thì các client khác sẽ nhận được: "abc: xin chào" hoặc có thể thêm thời gian vào trước ví dụ: **"2023/05/06 11:00:00PM abc: xin chào"**.

Hình ảnh chạy chương trình:

```
BT1905 > C chat_server.c > remove_client(int)
189 void broadcast_message(const char *msg, int sender_socket) {
190     for (int i = 0; i < num_clients; i++) {
191         if (clients[i].socket != sender_socket && clients[i].is_logged_in) {
192             send(clients[i].socket, msg, strlen(msg), 0);
193         }
194     }
195     pthread_mutex_unlock(&clients_mutex);
196 }
197
198 void remove_client(int socket) {
199     pthread_mutex_lock(&clients_mutex);
200     for (int i = 0; i < num_clients; i++) {
201         if (clients[i].socket == socket) {
202             close(socket);
203             for (int j = i; j < num_clients - 1; j++) {
204                 clients[j] = clients[j + 1];
205             }
206             num_clients--;
207         }
208     }
209 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang % gcc chat_server.c -o chat_server

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang % ./chat_server

Chat Server is listening on port 8080...

New connection accepted. Socket fd: 4

BT1905 % ./chat_server

2026/05/25 09:17:27PM bt1905: xin chào

2026/05/25 09:17:54PM 20235441: chào mọi người

New connection accepted. Socket fd: 6

2026/05/25 09:18:45PM 123: hello

2026/05/25 09:18:57PM 20235441: xin chào

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang % BT1905 % nc 127.0.0.1 8080

Vui lòng nhập tên theo cú pháp 'client_id: client_name'

20235441: Sai cú pháp! Thu lại.

Hệ thống: Vui lòng nhập tên theo cú pháp 'client_id: client_name'

20235441: thutrang

Chào mừng thutrang (ID: 20235441) đến với phòng chat!

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang % BT1905 % nc 127.0.0.1 8080

Vui lòng nhập tên theo cú pháp 'client_id: client_name'

20235441: trang

ID này đã có người sử dụng! Vui lòng chọn ID khác.

Vui lòng nhập tên theo cú pháp 'client_id: client_name'

bt1905: LapTrinhMang

Chào mừng LapTrinhMang (ID: bt1905) đến với phòng chat!

xin chào

2026/05/25 09:17:54PM 20235441: chào mọi người

Hệ thống: ntt (ID: 123) đã tham gia phòng chat.

2026/05/25 09:18:45PM 123: hello

2026/05/25 09:18:57PM 20235441: xin chào

Giải thích:

Chương trình gồm 2 thành phần:

- **Server (chat_server.c):** Quản lý nhiều client bằng multithread
- **Client:** Chạy nc 172.0.0.1 8080

Sau khi chạy chương trình:

- Server khởi động: *Chat Server is listening on port 8080...*
- Client kết nối:
 - *New connection accepted. Socket fd: ...*
 - Trên Client hiển thị: *Vui lòng nhập tên theo cú pháp 'client_id: client_name'*
- Client A nhập:

- 20235441 => Client A: *He thong: Sai cu phap! Thu lai.*
Vui long nhap ten theo cu phap 'client_id: client_name'
- 20235441: thutrang
=> Client A: *Chao mung thutrang (ID: 20235441) den voi phong chat!*
- Client B nhập:
 - 20235441: trang
=> Client B: *ID nay da co nguoi su dung! Vui long chon ID khac.*
Vui long nhap ten theo cu phap 'client_id: client_name'
 - bt1905: LapTrinhMang
=> Client B: *Chao mung LapTrinhMang (ID: bt1905) den voi phong chat!*
=> Client A: *He thong: LapTrinhMang (ID: bt1905) da tham gia phong chat.*
- Khi client A, B chat:
 - Client B chat: xin chao
=> Server: *2026/05/25 09:17:27PM bt1905: xin chao*
=> Client A: *2026/05/25 09:17:27PM bt1905: xin chao*
 - Client A chat: chao mọi người
=> Server: *2026/05/25 09:17:54PM 20235441: chao mọi người*
=> Client A: *2026/05/25 09:17:54PM 20235441: chao mọi người*
- Client C gia nhập phòng chat sau:
 - 123: ntt
=> Server: *New connection accepted. Socket fd: 6*
=> Client A, B: *He thong: ntt (ID: 123) da tham gia phong chat*
 - Client C chat: hello
=> Server: *2026/05/25 09:18:45PM 123: hello*
=> Client A, B: *2026/05/25 09:18:45PM 123: hello*

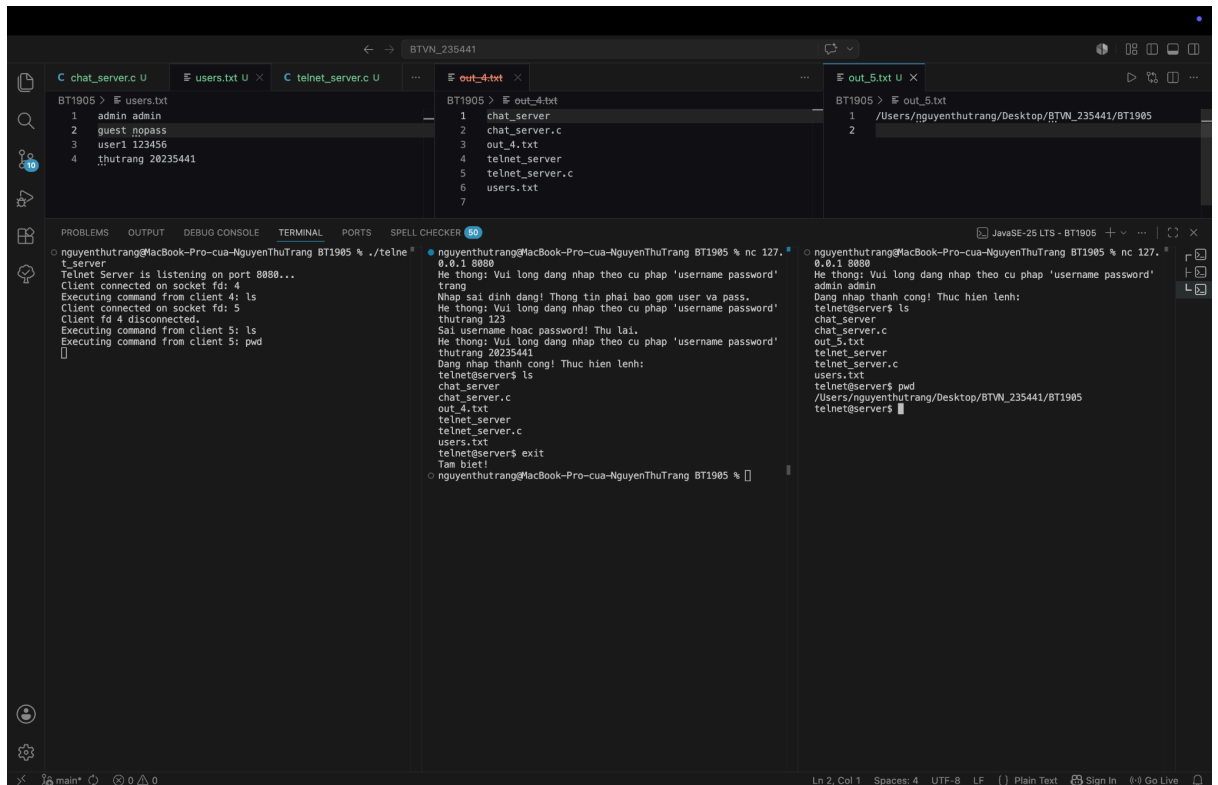
Bài 2: Telnet Server

Đề bài:

Viết chương trình telnet_server thực hiện các chức năng sau:

- Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass ví dụ:
admin admin
guest nopass
...
- Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.
- Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client.
- Dùng hàm system("dir > out.txt") để thực hiện lệnh, dir là ví dụ lệnh dir mà client gửi. > out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

Hình ảnh chạy chương trình:



Giải thích:

Chương trình gồm 2 thành phần:

- **Server (telnet_server.c):** Quản lý nhiều client bằng multithread
- **Client:** Chạy nc 172.0.0.1 8080

Sau khi chạy chương trình:

- Server khởi động: *Telnet Server is listening on port 8080...*
- Client kết nối:
 - *Client connected on socket fd: ...*
 - Trên Client hiển thị:
He thong: Vui long dang nhap theo cu phap 'username password'
- Client A nhập:
 - trang
=> Client A:
Nhap sai dinh dang! Thong tin phai bao gom user va pass.
He thong: Vui long dang nhap theo cu phap 'username password'
 - thutrang 123
=> Client A:
Sai username hoac password! Thu lai.
He thong: Vui long dang nhap theo cu phap 'username password'
 - thutrang 20235441
=> Client A:
Dang nhap thanh cong! Thuc hien lenh:
telnet@server\$
 - Trong client A nhập lệnh:
 - *telnet@server\$ ls*: Liệt kê danh sách các tệp tin và thư mục nằm trong mục BT1905

=> Server:

Executing command from client 4: ls

=> Client A:

```
telnet@server$ ls
chat_server
chat_server.c
out_4.txt
telnet_server
telnet_server.c
users.txt
telnet@server$
```

- *telnet@server\$ exit*: Client gõ "exit" thoát khỏi phiên làm việc và dọn dẹp file *out_5.txt* sau khi client A ngắt kết nối

=> Server:

Executing command from client 4: exit

=> Client A:

Tam biệt!

- Client B kết nối: => Server: *Client connected on socket fd: 5*

- admin admin

=> Client B:

Dang nhap thanh cong! Thuc hien lenh:

```
telnet@server$
```

- Trong client B nhập lệnh:

- *telnet@server\$ ls*

=> Server:

Executing command from client 5: ls

=> Client B:

```
telnet@server$ ls
chat_server
chat_server.c
out_5.txt
telnet_server
telnet_server.c
users.txt
telnet@server$
```

- *telnet@server\$ pwd*: hiển thị đường dẫn đầy đủ (đường dẫn tuyệt đối) của thư mục và sẽ thay thế kết quả của lệnh trước ở file *out_5.txt*

=> Server:

Executing command from client 5: pwd

=> Client B:

/Users/nguyenthutrang/Desktop/BTVN_235441/BT1905

Bài 3: Time Server

Đề bài:

Lập trình ứng dụng *time_server* thực hiện chức năng sau:

- Chấp nhận nhiều kết nối từ các client sử dụng kỹ thuật multiprocessing.
- Client gửi lệnh GET_TIME [format] để nhận thời gian từ server.
- format là định dạng thời gian server trả về client. Các format cần hỗ trợ gồm:
 - dd/mm/yyyy – vd: 30/01/2023
 - dd/mm/yy – vd: 30/01/23
 - mm/dd/yyyy – vd: 01/30/2023
 - mm/dd/yy – vd: 01/30/23
- Cần kiểm tra tính đúng đắn của lệnh client gửi lên server.

Hình ảnh chạy chương trình:

The screenshot shows a VS Code editor with the following content:

server.log

```

1 [2026-05-25 22:44:57] [127.0.0.1:61676] [CONNECTED] Client connected to server
2 [2026-05-25 22:45:02] [127.0.0.1:61677] [CONNECTED] Client connected to server
3 [2026-05-25 22:46:14] [127.0.0.1:61676] [ERROR] Failed Cmd: [gettime]
4 [2026-05-25 22:46:28] [127.0.0.1:61676] [ERROR] Failed Cmd: [GET_TIME]
5 [2026-05-25 22:47:48] [127.0.0.1:61677] [DISCONNECTED] Client disconnected
6 [2026-05-25 22:48:18] [127.0.0.1:61676] [SUCCESS] Cmd: [GET_TIME dd/mm/yyyy] -> Resp: [25/05/2026]
7 [2026-05-25 22:48:36] [127.0.0.1:61676] [SUCCESS] Cmd: [GET_TIME mm/dd/yy] -> Resp: [05/25/26]
8

```

Terminal Output:

```

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 % gcc tim_e_server.c -o time_server -lpthread
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 % ./time_server
Server
Time Server is listening on port 8080...
New connection accepted from 127.0.0.1:61676 (fd 4)
New connection accepted from 127.0.0.1:61677 (fd 5)
Client 127.0.0.1:61677 trên socket fd 5 đã ngắt kết nối.
[]

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 % nc 127.0.0.1 8080
Kết nối thành công. Hãy gửi lệnh 'GET_TIME [format]'
gettime
Lỗi: Sai cú pháp lệnh hoặc định dạng không được hỗ trợ!
GET_TIME
GET_TIME dd/mm/yyyy
25/05/2026
GET_TIME mm/dd/yy
05/25/26
[]

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 % nc 127.0.0.1 8080
Kết nối thành công. Hãy gửi lệnh 'GET_TIME [format]'
^C
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 %

```

Giải thích:

Chương trình gồm 2 thành phần:

- **Server (time_server.c):** Quản lý nhiều client bằng multithread
- **Client:** Chạy nc 127.0.0.1 8080

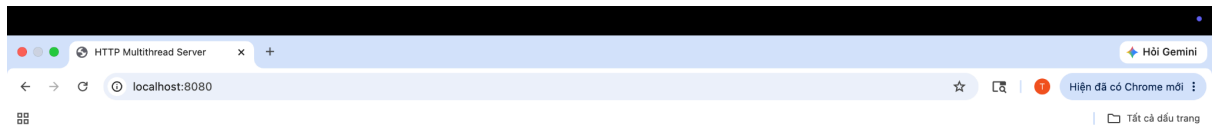
Sau khi chạy chương trình:

- Server khởi động: *Time Server is listening on port 8080...*
- Client kết nối:
 - New connection accepted from 127.0.0.1:61676 (fd 4)*
 - New connection accepted from 127.0.0.1:61677 (fd 5)*
- Client A nhập: gettime
=> server.log:

```

[2026-05-25 22:46:14] [127.0.0.1:61676] [ERROR] Failed Cmd:
[gettime]

```

Xin chào các bạn!

Giải thích:

Chương trình:

- **Server (http_server.c):** xử lý nhiều trình duyệt truy cập cùng một lúc mà không bị nghẽn (ví dụ khi một file HTML đang được tải).

Sau khi chạy chương trình:

- Server khởi động: *HTTP Server đa luồng đang chạy tại http://localhost:8080 ...*
- Nhập địa chỉ: <http://localhost:8080> trên trình duyệt

=> Terminal của server, sẽ thấy toàn bộ nội dung HTTP Request:

Luồng xử lý đa luồng (Multipthread). Các dòng log xen kẽ nhau:

- [Server] Có kết nối mới, Socket fd: 4
- --- HTTP REQUEST FROM CLIENT (fd 4) ---
- Request from client 4...: Tiến trình **con** nhận và in dữ liệu.
- Child process handled and closed client 4: Tiến trình **con** sau khi gửi phản hồi xong thì tự đóng socket và kết thúc (exit(0)).

Khi truy cập <http://localhost:8080>, trình duyệt gửi một "lá thư" yêu cầu:

- GET / HTTP/1.1:
 - GET: Phương thức yêu cầu lấy dữ liệu.
 - /: Đường dẫn (trang chủ).
 - HTTP/1.1: Phiên bản giao thức.
- Host: localhost:8080: Địa chỉ mà trình duyệt đang nhắm tới.
- User-Agent: "Chứng minh thư" của trình duyệt.

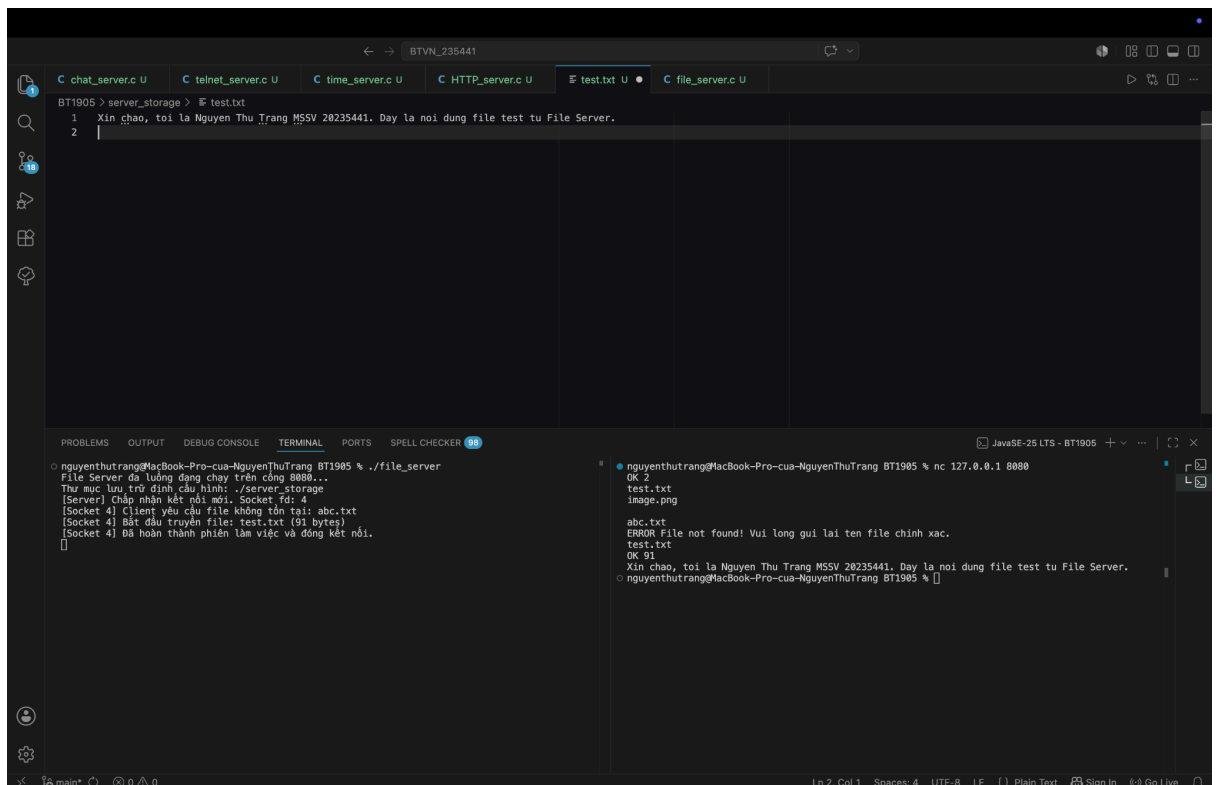
Bài 5: File Server

Đề bài:

Lập trình ứng dụng file server đơn giản như sau:

- Khi client mới được chấp nhận, server sẽ gửi danh sách các file cho client. Các file này trong thư mục được thiết lập trên server. Dòng đầu danh sách là chuỗi “OK N\r\n” nếu có N file trong thư mục. Các tên file phân cách nhau bởi ký tự \r\n, kết thúc danh sách bởi ký tự \r\n\r\n.
- Nếu không có file nào trong thư mục thì server gửi chuỗi “ERROR No files to download\r\n” rồi đóng kết nối.
- Client gửi tên file để tải về, server kiểm tra nếu file tồn tại thì gửi chuỗi “OK N\r\n” trong đó N là kích thước file, sau đó là nội dung file cho client và đóng kết nối, nếu file không tồn tại thì gửi thông báo lỗi và yêu cầu client gửi lại tên file.

Hình ảnh chạy chương trình:



```
BT1905 > server_storage > test.txt
1 Xin chào, tôi là Nguyen Thu Trang MSSV 20235441. Đây là nội dung file test tu File Server.
2

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 % ./file_server
File Server đã luồng đang chạy trên cổng 8080...
Thư mục lưu trữ định cấu hình: ./server_storage
[Server] Chấp nhận kết nối mới. Socket fd: 4
[Socket 4] Client yêu cầu file không tồn tại: abc.txt
[Socket 4] Bắt đầu truyền file: test.txt (91 bytes)
[Socket 4] Đã hoàn thành phiên làm việc và đóng kết nối.

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 % nc 127.0.0.1 8080
OK 2
test.txt
image.png
abc.txt
ERROR File not found! Vui lòng gửi lại tên file chính xác.
test.txt
OK 91
Xin chào, tôi là Nguyen Thu Trang MSSV 20235441. Đây là nội dung file test tu File Server.
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BT1905 %
```

Giải thích:

Chương trình gồm 2 thành phần:

- **Server (file_server.c):** Quản lý nhiều client bằng multithread
- **Client:** Chạy nc 127.0.0.1 8080

Use case:

- Server khởi động:
File Server đã luồng đang chạy trên cổng 8080...
Thư mục lưu trữ định cấu hình: ./server_storage
- Client kết nối: *[Server] Chấp nhận kết nối mới. Socket fd: 4*

- Client A:
 - OK 2*
 - test.txt*
 - image.png*
- Client A nhập: abc.txt
 - => Server:
 - [Socket 4] Client yêu cầu file không tồn tại: abc.txt*
 - [Socket 4] Bắt đầu truyền file: test.txt (91 bytes)*
 - [Socket 4] Đã hoàn thành phiên làm việc và đóng kết nối.*
 - => Client A:
 - ERROR File not found! Vui lòng gửi lại tên file chính xác.*
- Client A nhập: test.txt
 - => Server:
 - [Socket 4] Bắt đầu truyền file: test.txt (91 bytes)*
 - [Socket 4] Đã hoàn thành phiên làm việc và đóng kết nối.*
 - => Client A:
 - OK 91*
 - Xin chào, tôi là Nguyễn Thu Trang MSSV 20235441. Đây là nội dung file test từ File Server.*

Bài 6: Pair Chat Server

Đề bài:

Sử dụng kỹ thuật đa luồng, lập trình ứng dụng chat nhóm 2 người như sau:

- Mỗi client khi kết nối đến server sẽ được lưu vào hàng đợi, nếu số lượng client trong hàng đợi là 2 thì 2 client này sẽ được ghép cặp với nhau.
- Khi 2 client đã được ghép cặp thì tin nhắn gửi từ client này sẽ được chuyển tiếp sang client kia và ngược lại.
- Nếu 1 trong 2 client ngắt kết nối thì client còn lại cũng được ngắt kết nối.

Hình ảnh chạy chương trình:

The screenshot shows a code editor with a C file named `pair_chat_server.c`. The code implements a multithreaded server that listens on port 8080. It uses a queue to store clients waiting for a partner. When two clients are in the queue, they are paired, and their messages are forwarded to each other. The code includes comments in Vietnamese explaining the logic.

```
115 while (1) {
116     int len = recv(my_sock, buf, sizeof(buf) - 1, 0);
117
118     // Nếu 1 trong 2 client ngắt kết nối (len <= 0)
119     if (len <= 0) {
120         printf("(Ngắt kết nối) Client fd %d rời phòng.\n", my_sock);
121         char *bye_msg = "He thông: Ban chat của bạn đã rời phòng. Ket noi dong.\n";
122         send(peer_sock, bye_msg, strlen(bye_msg), 0);
123         close(my_sock);
124         close(peer_sock);
125         break;
126     }
127
128     buf[len] = '\0';
129     char send_buf[BUF_SIZE + 30];
130     snprintf(send_buf, sizeof(send_buf), "Friend: %s", buf);
131     send(peer_sock, send_buf, sizeof(send_buf), 0);
132 }
```

The terminal output shows the server running and the interaction between two clients (BT1905 and BT1905) who are paired and chat with each other. The messages are forwarded between them, showing the "Friend" prefix.

Giải thích:

Chương trình gồm 2 thành phần:

- **Server (pair_chat_server.c):** Quản lý nhiều client bằng multithread
- **Client:** Chạy nc 127.0.0.1 8080

Use case:

- Server khởi động:
Pair Chat Server đang chạy trên cổng 8080...
Đang chờ người dùng kết nối để thực hiện ghép cặp...
- Client kết nối: *[Hàng đợi] Client fd ... đang đợi bạn chat...*
- Client A:
Đang tìm kiếm bạn chat... Vui lòng đợi trong giây lát.
- Client B kết nối thì:
=> Server:
[Ghép cặp] Thành công! Cặp đôi: (fd 4 <---> fd 5)
=> Client B:

He thống: Da tìm thay ban chat! Phong chat 2 nguoi bat dau. Hay go tin nhan.

- Client C kết nối:

=> Server:

[Hàng đợi] Client fd 6 đang đợi bạn chat...

=> Client C:

Dang tìm kiếm bạn chat... Vui lòng đợi trong giây lát.

- Client A, B có thể chat trực tiếp khi một trong 2 kết nối thì cả 2 đều mất

=> Server: *[Ngắt kết nối] Client fd 5 rời phòng.*

[Ngắt kết nối] Client fd 4 rời phòng.